THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

PROJECT – LOW LEVEL DESIGN

On

SecureVault – Hardened Cloud Infrastructure for E-Commerce Data

Secure File Storage & AES-256-GCM Encryption Platform

NAME : Sahil Ameen **REG NO :** 23BCCE130	**GUIDED BY :** Ms. Vijayalaxmi

TABLE OF CONTENTS

- Introduction……... 1
  - Purpose and Scope…….. 2
  - Project Executive Summary---------- 2
  - Constraints ……..2
  - Document Organization---------- 3
- System Overview……... 3
- Module Design----------4
  - Crypto Engine Module---------- 4
  - Key Derivation Module---------- 4
  - Vault Storage Module---------- 5
  - Secure Delete Module---------- 5
  - Web Portal and Dashboard Module---------- 5
- Data Flow Design---------- 5
- Database Design---------- 5
  - Vault Metadata Table---------- 5
  - .agv File Format Table---------- 6
  - Function Interface Table---------- 6
- User Interface Design---------- 6
- Security Implementation---------- 6
- Testing and Validation---------- 7
- Project References---------- 7
- Glossary---------- 7
- Front-end---------- 8
- Back-end---------- 9

1. Introduction

SecureVault is a Django-based, Python-powered secure file storage platform designed for the e-commerce domain. It implements AES-256-GCM (Advanced Encryption Standard – Galois/Counter Mode) symmetric encryption combined with PBKDF2-HMAC-SHA256 key derivation to protect sensitive business files including invoices, customer data, payment records, and vendor contracts stored on the server.

The system directly addresses the critical industry challenge of Data-at-Rest protection — ensuring that even if a server's storage is accessed or compromised, the files within remain completely unreadable to an attacker without the correct encryption password.

1.1 Purpose and Scope

The purpose of this Low Level Design (LLD) document is to provide a detailed technical specification of the SecureVault system's internal components, module-level interfaces, database schema, data flow pipelines, and implementation logic. This document serves as the direct implementation blueprint for developers.

The scope of this LLD covers:

- Detailed design of the Crypto Engine, Key Derivation, Vault Storage, Secure Delete, and Web Portal modules.
- Internal function signatures, input/output contracts, and error-handling logic for each module.
- Database schema design including table definitions, field types, constraints, and indexing strategy.
- Step-by-step data flow for file encryption (upload) and decryption (download) operations.
- User Interface wireframe descriptions for all seven portal pages.
- Security implementation details covering OWASP, PCI-DSS, and GDPR requirements.
- Unit, integration, and security test cases with expected outcomes.

Out of scope: deployment infrastructure, CI/CD pipelines, and cloud-provider-specific configuration.

1.2 Project Executive Summary

SecureVault solves the critical data-at-rest security problem for e-commerce businesses. Sensitive files such as invoices, customer records, and payment data are encrypted using AES-256-GCM before being written to disk. The system guarantees that even in the event of server compromise, stored files remain cryptographically unreadable. The platform is built on Django 4.2 with Python's cryptography library, features a full audit log, and supports role-based multi-user isolation.

1.3 Constraints

Constraint	Description
File Size Limit	Default Django DATA_UPLOAD_MAX_MEMORY_SIZE: 2.5 MB. Extendable to 100+ MB via chunked streaming.
Encryption Standard	AES-256-GCM only (NIST FIPS 197). No legacy ciphers permitted.
Key Derivation	PBKDF2-HMAC-SHA256 with minimum 480,000 iterations (OWASP 2023).
Password Policy	Passwords are never stored. Re-entry required for every decrypt operation.
Database	SQLite for development; PostgreSQL required for production multi-user deployment.
Concurrency	Django dev server: single-threaded. Production: Gunicorn + Nginx required.
OS Dependency	os.urandom() for salt/nonce generation; requires a system CSPRNG.

1.4 Document Organization

This document is organized as follows: Section 2 provides a System Overview. Section 3 details the Module Design for each of the five core modules. Section 4 describes the Data Flow Design. Section 5 covers the Database Design including schema tables. Section 6 documents the User Interface Design. Section 7 explains Security Implementation. Section 8 outlines Testing and Validation. Section 9 lists Project References. Section 10 provides a Glossary. Sections 11 and 12 cover the Front-end and Back-end designs respectively.

2. System Overview

SecureVault acts as a security middleware layer between e-commerce business users and local server file storage. It intercepts sensitive files at upload, encrypts them using AES-256-GCM before they are written to disk, and stores only encrypted ciphertext. The system is structured as a Django Model-View-Template (MVT) application with five discrete internal modules.

Technology Stack:

Component	Technology / Details
Backend Framework	Django 4.2 (Python) – MVT architecture, ORM, session management, CSRF protection.
Encryption Library	Python cryptography >= 41.0 – AESGCM, PBKDF2HMAC, hashes.SHA256, InvalidTag.
File Storage	Django FileField with MEDIA_ROOT – encrypted .vault blobs on local disk.
Database	SQLite (dev) / PostgreSQL (prod) – managed via Django ORM.
Frontend	HTML5 + CSS3 – Dark industrial UI with Space Mono and Syne Google Fonts.
Authentication	Django built-in Auth – session-based login, CSRF middleware.
Key Derivation	PBKDF2-HMAC-SHA256 – 480,000 iterations, 256-bit key, 256-bit random salt.
Vault Format	Custom .vault: MAGIC(8B) + SALT(32B) + NONCE(12B) + CIPHERTEXT(N+16B).

3. Module Design

SecureVault is decomposed into five core modules, each with a clearly defined responsibility, internal functions, and interface contracts.

3.1 Crypto Engine Module

File: vault/encryption.py

Responsibility: Provides all AES-256-GCM encryption and decryption operations. This module is purely functional with no Django ORM dependency and is the cryptographic core of the system.

Function	Signature	Description
encrypt_file_to_bytes()	encrypt_file_to_bytes(plaintext: bytes, password: str) -> (bytes, bytes, bytes)	Generates salt and nonce, derives key via PBKDF2, encrypts plaintext with AES-GCM. Returns (vault_blob, salt, nonce).
decrypt_file_from_bytes ()	decrypt_file_from_bytes(vault_blob: bytes, password: str) -> bytes	Reads MAGIC header, extracts salt and nonce, re-derives key, decrypts and verifies GCM tag. Raises InvalidTag on failure.
derive_key()	derive_key(password: str, salt: bytes) -> bytes	Executes PBKDF2-HMAC-SHA256 with 480,000 iterations, returning 32-byte (256-bit) AES key.
get_encryption_info()	get_encryption_info() -> dict	Returns metadata dict: algorithm='AES-256-GCM', kdf='PBKDF2-HMAC-SHA256', iterations=480000, key_size=256.

Error Handling: derive_key() propagates ValueError on empty password. decrypt_file_from_bytes() catches cryptography.exceptions.InvalidTag and raises it to the view layer for user-facing error display.

3.2 Key Derivation Module

Embedded within vault/encryption.py as the derive_key() function.

Responsibility: Transforms a human-readable password string into a strong 256-bit AES key using a computationally expensive key stretching algorithm. Prevents brute-force and dictionary attacks by introducing deliberate computational cost.

Algorithm Details:

Parameter	Value	Rationale
Algorithm	PBKDF2-HMAC-SHA256	NIST-approved and OWASP-recommended KDF for password hashing.
Iterations	480,000	Meets OWASP 2023 minimum; ~0.3-0.5s per operation on modern hardware.
Salt Length	32 bytes (256-bit)	Cryptographically random via os.urandom(32). Unique per file.
Key Output	32 bytes (256-bit)	Directly used as the AES-256-GCM encryption key.
Salt Storage	BinaryField in DB	Salt stored; password never stored. Re-derivation on demand.

3.3 Vault Storage Module

File: vault/views.py (upload_file view) + Django FileField

Responsibility: Manages the storage lifecycle of encrypted .vault blobs — receiving the encrypted bytes from the Crypto Engine and persisting them to the MEDIA_ROOT/encrypted/ directory via Django's FileField.

The .vault binary format is structured as follows:

Offset	Field	Size	Description
0	MAGIC	8 bytes	Header signature: b'SECRVLT1' for format validation.
8	SALT	32 bytes	256-bit PBKDF2 salt (unique per file).
40	NONCE	12 bytes	96-bit AES-GCM nonce (unique per file).
52	CIPHERTEXT	N + 16 bytes	AES-256-GCM encrypted data with embedded 128-bit auth tag.

On upload, the view calls encrypt_file_to_bytes(), wraps the result in a ContentFile, and saves it via Django's FileField. The original plaintext is never written to disk at any point.

3.4 Secure Delete Module

File: vault/views.py (delete_file view)

Responsibility: Permanently removes an encrypted .vault file from both the filesystem and the database, with full audit trail generation.

Delete Operation Sequence:

- Step 1 – Authorization check: verify request.user == encrypted_file.user.
- Step 2 – Physical file removal: call encrypted_file.encrypted_file.delete(save=False) to remove the .vault blob from MEDIA_ROOT/encrypted/.
- Step 3 – Database record deletion: call encrypted_file.delete() to remove the EncryptedFile ORM record.
- Step 4 – Audit log: create AuditLog entry with action='delete', file reference SET_NULL, and timestamp.

Note: The delete operation is irreversible. No recycle bin or soft-delete mechanism is implemented. The GCM authentication tag and ciphertext are unrecoverable after physical deletion.

3.5 Web Portal and Dashboard Module

Files: vault/views.py, vault/templates/

Responsibility: Provides the full Django-rendered HTML interface across seven pages, handling all user-facing interactions including file upload with drag-and-drop, per-file decrypt and delete actions, audit log display, and dashboard statistics.

View-to-Template Mapping:

Django View	URL Pattern	Template	Function
register()	/register/	register.html	New user account creation with hashed password.
login_view()	/login/	login.html	Credential authentication via Django auth backend.
logout_view()	/logout/	Redirect	Session invalidation and redirect to login.
dashboard()	/dashboard/	dashboard.html	Display files, statistics, recent audit

Django View	URL Pattern	Template	Function
			events.
upload_file()	/upload/	upload.html	Drag-and-drop upload, password entry, encryption trigger.
download_file(id)	/download/<id>/	download.html	Password entry, decryption, file streaming.
delete_file(id)	/delete/<id>/	confirm_delete.html	Deletion confirmation dialog and execution.
audit_log()	/audit/	audit_log.html	Full security event log for authenticated user.

4. Data Flow Design

4.1 Encryption Flow (File Upload)

Step	Action	Component	Detail
1	User Input	Browser → Django View	HTTPS POST with multipart form: file bytes + password string.
2	Salt Generation	Crypto Engine	os.urandom(32) → 256-bit cryptographically random salt.
3	Key Derivation	Key Derivation Module	PBKDF2HMAC(SHA256, 480000 iterations) → 256-bit AES key.
4	Nonce Generation	Crypto Engine	os.urandom(12) → 96-bit AES-GCM nonce.
5	AES-256-GCM Encrypt	Crypto Engine	AESGCM(key).encrypt(nonce, plaintext, None) → ciphertext + 128-bit auth tag.
6	Pack .vault Blob	Vault Storage Module	MAGIC(8B) + SALT(32B) + NONCE(12B) + CIPHERTEXT packed.
7	Save to Disk	Django FileField	.vault blob written to MEDIA_ROOT/encrypted/. Plaintext discarded.
8	Record Metadata	Django ORM	EncryptedFile record: filename, salt, nonce, file_size. Password never stored.

4.2 Decryption Flow (File Download)

Step	Action	Component	Detail
1	Password Entry	Browser → Django View	HTTPS POST with file ID and user-supplied password.
2	Fetch from Disk	Vault Storage Module	Django reads .vault blob from MEDIA_ROOT/encrypted/ via FileField path.
3	Unpack Header	Crypto Engine	Reads MAGIC(8B) for validation, extracts SALT(32B) and NONCE(12B).
4	Re-derive Key	Key Derivation Module	PBKDF2-HMAC-SHA256 with supplied password + stored salt → identical 256-bit key.
5	AES-256-GCM Decrypt	Crypto Engine	AESGCM(key).decrypt() verifies auth tag + returns plaintext. InvalidTag on wrong password.
6	Stream to Browser	Django View	Django HttpResponse with Content-Disposition: attachment; original filename restored.

5. Database Design

5.1 Vault Metadata Table (EncryptedFile)

Django model: vault/models.py → class EncryptedFile. Stores all metadata for each encrypted file record.

Field	Type	Constraints	Description
id	AutoField (INT)	PK, Auto-increment	Unique identifier for each encrypted file record.
user	ForeignKey(User)	NOT NULL, CASCADE	Owner of the file. Enforces per-user data isolation.
original_filename	VARCHAR(255)	NOT NULL	Original filename before encryption (e.g., orders.csv).
encrypted_file	FileField	NOT NULL, UNIQUE	Relative path to .vault blob under MEDIA_ROOT/encrypted/.
file_type	VARCHAR(50)	NOT NULL, CHOICES	Category: invoice / customer_data / payment_info / contract / report / other.
file_size	BigIntegerField	NOT NULL, >= 0	Original file size in bytes before encryption.
encryption_algorithm	VARCHAR(50)	NOT NULL, DEFAULT	Always 'AES-256-GCM'. Stored for audit and display.
salt	BinaryField(32)	NOT NULL	256-bit PBKDF2 salt. Required for key re-derivation on decrypt.
nonce	BinaryField(12)	NOT NULL	96-bit AES-GCM nonce. Required for decryption.
uploaded_at	DateTimeField	NOT NULL, auto_now_add	UTC timestamp of record creation.
description	TextField	NULLABLE, BLANK	Optional user-provided notes about the file.
download_count	PositiveIntegerField	NOT NULL, DEFAULT=0	Counts successful decrypt/download events.

5.2 .vault File Format Table

Byte Offset	Field Name	Size	Data Type	Description
0–7	MAGIC	8 bytes	ASCII bytes	b'SECRVLT1' — file format signature for validation.
8–39	SALT	32 bytes	Raw binary	256-bit PBKDF2 salt, generated via os.urandom(32).
40–51	NONCE	12 bytes	Raw binary	96-bit AES-GCM nonce, generated via os.urandom(12).
52–(N+67)	CIPHERTEXT	N+16 bytes	Raw binary	AES-256-GCM ciphertext + embedded 128-bit GCM auth tag.

5.3 Function Interface Table

Function	Module	Input	Output	Exception
encrypt_file_to_bytes()	encryption.py	plaintext: bytes, password: str	(vault_blob: bytes, salt: bytes, nonce: bytes)	ValueError on empty password
decrypt_file_from_bytes ()	encryption.py	vault_blob: bytes, password: str	plaintext: bytes	InvalidTag on wrong password/tampered file
derive_key()	encryption.py	password: str, salt: bytes	key: bytes (32)	ValueError on empty password
upload_file()	views.py	request: HttpRequest (POST)	HttpResponseRedirect to dashboard	ValidationError, IOError
download_file(id)	views.py	request: HttpRequest, id: int	HttpResponse (file stream)	Http404, InvalidTag
delete_file(id)	views.py	request: HttpRequest, id: int	HttpResponseRedirect to dashboard	Http404, PermissionDenied

6. User Interface Design

All pages use a consistent dark industrial design language: animated grid background, Space Mono monospace font for technical data, Syne font for headings, and a teal/dark color palette. Django server-side renders all templates. JavaScript is used only for UI enhancements (drag-and-drop, toast notifications, password strength meter).

Page	Template	Key UI Elements
Login	login.html	Email + password fields, CSRF token, 'Forgot password' placeholder, dark card layout.
Register	register.html	Username, email, password + confirm fields. Real-time password strength meter (green/amber/red indicator bar).
Dashboard	dashboard.html	Stats row: total files, total size (MB), total downloads. File table with type badges (Invoice, Payment Info, etc.), Decrypt and Delete action buttons per row. Recent audit events panel.
Upload	upload.html	Drag-and-drop zone with file info display (name, size). File type selector. Password + confirm password fields. Submit triggers encrypt-and-save pipeline.
Download	download.html	File info display (name, type, size, algorithm). Single password field. Submit triggers decrypt-and-stream pipeline. InvalidTag shows inline error toast.
Confirm Delete	confirm_delete.html	File info summary. Confirm and Cancel buttons. Confirm triggers permanent delete.
Audit Log	audit_log.html	Scrollable table: timestamp, action type, file name, IP address. Color-coded action badges (upload=blue, delete=red, failed_decrypt=orange).

7. Security Implementation

Security Control	Implementation Detail	Standard / Compliance
AES-256-GCM Encryption	Python cryptography library AESGCM class. 256-bit key, 128-bit auth tag, 96-bit nonce.	NIST FIPS 197, NIST SP 800-38D
PBKDF2 Key Derivation	PBKDF2HMAC(SHA256, 480,000 iterations, 32-byte salt, 32-byte key).	OWASP Password Storage 2023
Salt Uniqueness	os.urandom(32) per file. Prevents rainbow table attacks.	NIST SP 800-132
Nonce Uniqueness	os.urandom(12) per encrypt call. Prevents GCM nonce-reuse attack.	NIST SP 800-38D
No Password Storage	Passwords processed in-memory only. Never logged, stored, or included in any DB field.	PCI-DSS Req 3.4, GDPR Art 32
Integrity Verification	128-bit GCM authentication tag verified on every decrypt. Any bit-level tampering raises InvalidTag.	NIST SP 800-38D
CSRF Protection	Django CsrfViewMiddleware enforces CSRF token on all POST endpoints.	OWASP CSRF Prevention
SQL Injection Prevention	Django ORM parameterized queries for all DB operations.	OWASP SQL Injection Prevention
Session Security	Server-side sessions only (django_session table). SESSION_COOKIE_HTTPONLY=True. SESSION_COOKIE_SECURE=True in prod.	OWASP Session Management
Audit Logging	All events (upload, download, delete, login, logout, failed_decrypt) logged with user, IP, timestamp, details.	PCI-DSS Req 10, GDPR Art 30
User Data Isolation	All EncryptedFile queries filtered by request.user. No cross-user access possible at ORM level.	OWASP Broken Access Control
In-Memory Processing	File plaintext only exists in RAM. Never written to any temporary file or disk path.	PCI-DSS Req 3.1

8. Testing and Validation

Test ID	Test Case	Input	Expected Output	Type
TC-01	Successful file encryption	Valid file + password	.vault blob saved to disk; DB record created	Unit
TC-02	Successful file decryption	Correct .vault blob + correct password	Original plaintext bytes returned; download_count incremented	Unit
TC-03	Wrong password rejection	Correct .vault blob + wrong password	InvalidTag raised; user sees 'Incorrect password' error	Unit
TC-04	Tampered vault file rejection	Modified .vault blob + correct password	InvalidTag raised; decrypt fails	Security
TC-05	MAGIC header validation	.vault blob with corrupted MAGIC bytes	ValueError or format error raised before KDF	Unit
TC-06	Salt uniqueness per file	Two identical files + same password	Two different .vault blobs (different ciphertexts)	Unit
TC-07	Password not in DB	Upload any file	No password field in EncryptedFile DB record	Security
TC-08	CSRF token enforcement	POST /upload/ without CSRF token	403 Forbidden response	Security
TC-09	Cross-user file access	User B requests User A's file ID	404 Not Found (ORM filter enforced)	Security
TC-10	Audit log on failed decrypt	Wrong password on download	AuditLog entry: action='failed_decrypt' with IP and timestamp	Integration
TC-11	Secure delete removes file	Confirmed delete action	.vault blob absent from MEDIA_ROOT; DB record deleted	Integration
TC-12	Login required enforcement	Unauthenticated GET /dashboard/	302 redirect to /login/	Integration

9. Project References

Source	Reference
NIST FIPS 197	Advanced Encryption Standard (AES) – https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf
NIST SP 800-38D	GCM Mode of Operation – https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
OWASP Password Storage	Password Storage Cheat Sheet – https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
Python cryptography	Python Cryptography Library – https://cryptography.io/en/latest/
Django Documentation	Django 4.2 Official Docs – https://docs.djangoproject.com/en/4.2/
PCI-DSS v4.0	Payment Card Industry Data Security Standard – https://www.pcisecuritystandards.org/
GDPR Article 32	Security of Processing – https://gdpr-info.eu/art-32-gdpr/

10. Glossary

Term	Definition
AES-256-GCM	Advanced Encryption Standard with 256-bit key in Galois/Counter Mode — provides authenticated encryption with both confidentiality and integrity.
PBKDF2	Password-Based Key Derivation Function 2 — applies a pseudorandom function (here SHA-256) to a password with a salt over many iterations to produce a cryptographic key.
.vault	SecureVault's proprietary encrypted binary file format: MAGIC + SALT + NONCE + CIPHERTEXT.
GCM Tag	128-bit Galois/Counter Mode authentication tag appended to ciphertext — any tampering causes decryption failure (InvalidTag).
Salt	A 256-bit cryptographically random value generated per file using os.urandom(32) to ensure unique key derivation even for identical passwords.
Nonce	A 96-bit Number Used Once per AES-GCM encryption operation generated via os.urandom(12).
InvalidTag	Python cryptography library exception raised when the GCM authentication tag fails verification — indicates wrong password or tampered ciphertext.
MEDIA_ROOT	Django configuration setting for the local filesystem path where uploaded/encrypted files are stored.
MVT	Model-View-Template — Django's architectural pattern separating data (Model), logic (View), and presentation (Template) layers.
CSRF	Cross-Site Request Forgery — an attack mitigated by Django's built-in CsrfViewMiddleware and CSRF token in all POST forms.
Data-at-Rest	Refers to data stored on disk as opposed to data in transit over a network. SecureVault protects data-at-rest through AES-256-GCM encryption.
CSPRNG	Cryptographically Secure Pseudorandom Number Generator — used by os.urandom() to generate salts and nonces.

11. Front-End Design

The SecureVault front-end is a server-side rendered HTML5/CSS3 portal. Django's template engine handles all page rendering. No front-end framework (React, Vue) is used. JavaScript is used minimally for UX enhancements only.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}SecureVault{% endblock %} — E-Commerce File Security</title>

  {% block extra_css %}{% endblock %}
</head>
<body>

{% if user.is_authenticated %}
<nav>
  <a href="{% url 'dashboard' %}">
    <div>□</div>
    SecureVault
  </a>

  <div>
    <a href="{% url 'dashboard' %}">Dashboard</a>
    <a href="{% url 'upload_file' %}">Encrypt File</a>
    <a href="{% url 'audit_log' %}">Audit Log</a>
  </div>

  <div>
    <span>□  {{ user.username }}</span>
    <a href="{% url 'logout' %}">Logout</a>
  </div>
</nav>
{% endif %}

{% if messages %}
<div>
  {% for msg in messages %}
  <div>
    {% if msg.tags == 'success' %}✓{% elif msg.tags == 'error' %}✗{% else %}ℹ {% endif %}
    {{ msg }}
  </div>
  {% endfor %}
</div>

<script>
  setTimeout(() => {
    document.querySelectorAll('.message').forEach(m => {
      m.style.transition = 'opacity 0.5s';
      m.style.opacity = '0';
      setTimeout(() => m.remove(), 500);
    });
  }, 5000);
</script>
{% endif %}
```

```
<main>
    {% block content %}{% endblock %}
</main>

<footer>
    <span>SecureVault &copy; 2025 — E-Commerce Data Security Platform</span>
    <span>AES-256-GCM · PBKDF2-HMAC-SHA256 · Data-at-Rest Protection</span>
</footer>

</body>
</html>
```

11.1 Technology Stack

Technology	Role
HTML5	Semantic markup for all seven portal pages.
CSS3	Custom dark industrial theme with CSS Grid and Flexbox for layout.
Google Fonts	Space Mono (monospace, for technical data display) + Syne (headings).
Vanilla JavaScript	Drag-and-drop file zone, password strength meter, auto-dismiss toast notifications.
Django Templates	Server-side rendering with template inheritance via base.html layout.
CSRF Middleware	{% csrf_token %} tag in all POST forms for cross-site request forgery protection.

11.2 Page Design Details

Login Page (login.html)

Layout: centered dark card on animated grid background. Fields: username, password. CSRF token included. Failed login shows inline error message via Django messages framework. No JavaScript dependency for core auth flow.

Register Page (register.html)

Fields: username, email, password, confirm password. Real-time password strength indicator implemented in vanilla JS: reads password input on keyup, scores complexity (length >= 12, uppercase, lowercase, digit, special char), renders a colored bar and text label (Weak / Fair / Strong). CSRF token included.

Dashboard Page (dashboard.html)

Top statistics row displays: total encrypted files count, total storage used (formatted in KB/MB/GB), total successful downloads. Below: paginated file table with columns for filename, type badge, size, upload date, download count, and action buttons (Decrypt / Delete). Sidebar panel shows the five most recent AuditLog events with action icons.

Upload Page (upload.html)

Drag-and-drop zone implemented with JavaScript dragover, dragleave, and drop event listeners. On file drop or click-select, displays file name and size. File type dropdown selector. Encryption password field with confirmation. Submit button triggers Django POST to /upload/ which invokes the Crypto

Engine pipeline.

Download Page (download.html)

Displays file metadata (name, type, size, algorithm, upload date). Single password field. On correct password, Django streams the decrypted file as an HTTP attachment. On wrong password, view catches InvalidTag and re-renders the page with an inline error toast.

11.3 Front-End Security Controls

- All forms include Django's {% csrf_token %} to prevent Cross-Site Request Forgery.
- Password fields use type='password' to prevent browser autofill exposure.
- No encryption keys, salts, or nonces are exposed in the DOM or JavaScript.
- Content Security Policy (CSP) headers recommended for production deployment.
- All pages use HTTPS in production; Django SECURE_SSL_REDIRECT=True enforces this.

```
{% extends 'vault/base.html' %}
{% block title %}Dashboard{% endblock %}

{% block content %}
<div class="page-header">
  <div class="header-flex">
    <div>
      <h1 class="page-title">Vault Dashboard</h1>
      <p class="page-subtitle">
        // Welcome back, {{ user.first_name|default:user.username }}
      </p>
    </div>
    <a href="{% url 'upload_file' %}" class="btn btn-primary">
      + Encrypt New File
    </a>
  </div>
</div>

<!-- Stats -->
<div class="stats-grid">
  <div class="stat-card">
    <div class="stat-label">Total Files</div>
    <div class="stat-value">{{ stats.total_files }}</div>
    <div class="stat-desc">Encrypted & stored</div>
  </div>

  <div class="stat-card">
    <div class="stat-label">Total Size</div>
    <div class="stat-value stat-size">
      {{ stats.total_size_display }}
    </div>
    <div class="stat-desc">Original data protected</div>
  </div>

  <div class="stat-card">
    <div class="stat-label">Downloads</div>
    <div class="stat-value">{{ stats.total_downloads }}</div>
    <div class="stat-desc">Decryption operations</div>
  </div>
```

```
  <div class="stat-card">
    <div class="stat-label">Encryption</div>
    <div class="stat-value stat-small">AES-256</div>
    <div class="stat-desc">GCM authenticated mode</div>
  </div>
</div>

<div class="main-grid">
  <!-- Files -->
  <div>
    <a href="{% url 'upload_file' %}" class="upload-cta">
      <span>↑</span> Drag a file or click to encrypt →
    </a>

    <div class="card">
      <div class="section-header">
        <div class="section-title">□  Your Encrypted Files</div>
        <span class="badge badge-green">
          {{ stats.total_files }} files
        </span>
      </div>

      {% if files %}
        {% for f in files %}
        <div class="file-row">
          <div class="file-icon">
            {% if f.file_type == 'invoice' %}□
                        {% elif f.file_type == 'customer_data' %}□
                        {% elif f.file_type == 'payment_info' %}□
                        {% elif f.file_type == 'contract' %}□
                        {% elif f.file_type == 'report' %}□
                        {% else %}□{% endif %}
          </div>

          <div class="file-info">
            <div class="file-name" title="{{ f.original_filename }}">
              {{ f.original_filename }}
            </div>
            <div class="file-meta">
              {{ f.get_file_size_display }} ·
              {{ f.get_file_type_display }} ·
              {{ f.uploaded_at|date:"M d, Y H:i" }} ·
              {{ f.download_count }} download{{ f.download_count|pluralize }}
            </div>
          </div>

          <div class="file-actions">
            <span class="badge badge-green mono">AES-256-GCM</span>
            <a href="{% url 'download_file' f.id %}" class="btn-sm btn-sm-primary">
              ↓ Decrypt
            </a>
            <a href="{% url 'delete_file' f.id %}" class="btn-sm btn-sm-danger">
              □
                        </a>
          </div>
        </div>
        {% endfor %}
      {% else %}
        <div class="empty-state">
```

```
      <span class="empty-icon">□</span>
      No files yet. Encrypt your first file to get started.
    </div>
  {% endif %}
 </div>
</div>

<!-- Sidebar -->
<div>
 <!-- Encryption Info -->
 <div class="info-box card-accent">
  <div class="section-title">□  Encryption Config</div>

  <div class="enc-spec">
    <span class="enc-key">Algorithm</span>
    <span class="enc-val">{{ encryption_info.algorithm }}</span>
  </div>
  <div class="enc-spec">
    <span class="enc-key">Key Size</span>
    <span class="enc-val">{{ encryption_info.key_size }}</span>
  </div>
  <div class="enc-spec">
    <span class="enc-key">KDF</span>
    <span class="enc-val">PBKDF2-SHA256</span>
  </div>
  <div class="enc-spec">
    <span class="enc-key">Iterations</span>
    <span class="enc-val">480,000</span>
  </div>
  <div class="enc-spec">
    <span class="enc-key">Salt</span>
    <span class="enc-val">{{ encryption_info.salt_size }}</span>
  </div>
  <div class="enc-spec">
    <span class="enc-key">Authenticated</span>
    <span class="enc-val">✓ Yes (GCM)</span>
  </div>
 </div>

 <!-- Activity -->
 <div class="info-box">
  <div class="section-header">
    <div class="section-title">□  Recent Activity</div>
    <a href="{% url 'audit_log' %}" class="view-link">
      View all →
    </a>
  </div>

  {% if recent_logs %}
    {% for log in recent_logs %}
    <div class="log-item">
      <div class="log-dot {{ log.action }}"></div>
      <div>
        <div class="log-action">{{ log.get_action_display }}</div>
        {% if log.file %}
          <div class="log-time">
            {{ log.file.original_filename|truncatechars:25 }}
          </div>
        {% endif %}
```

```
          <div class="log-time">
            {{ log.timestamp|date:"M d H:i" }}
          </div>
        </div>
      </div>
      {% endfor %}
    {% else %}
      <p class="no-activity">No activity yet.</p>
    {% endif %}
  </div>
 </div>
</div>
{% endblock %}
```

12. Back-End Design

The SecureVault back-end is a Django 4.2 Python application following the MVT pattern. It handles all request routing, authentication, file I/O, encryption/decryption orchestration, database operations, and audit logging.

12.1 Django Project Structure

File / Directory	Purpose
securevault/settings.py	Core Django settings: INSTALLED_APPS, DATABASES, MEDIA_ROOT, SECRET_KEY, CSRF config, session settings.
securevault/urls.py	Root URL configuration routing all requests to the vault app.
vault/models.py	ORM models: EncryptedFile and AuditLog with all fields, constraints, and Meta ordering.
vault/views.py	Eight view functions handling register, login, logout, dashboard, upload, download, delete, and audit log.
vault/encryption.py	Crypto Engine: encrypt_file_to_bytes(), decrypt_file_from_bytes(), derive_key(), get_encryption_info().
vault/urls.py	App-level URL patterns mapping endpoints to view functions.
vault/templates/	Seven HTML/CSS templates: base.html, login.html, register.html, dashboard.html, upload.html, download.html, confirm_delete.html, audit_log.html.
vault/admin.py	Django admin site registration for EncryptedFile and AuditLog models.

Models.py

```
from django.db import models
from django.contrib.auth.models import User


class EncryptedFile(models.Model):
    FILE_TYPES = [
        ('invoice', 'Invoice'),
        ('customer_data', 'Customer Data'),
        ('payment_info', 'Payment Info'),
        ('contract', 'Contract'),
        ('report', 'Report'),
        ('other', 'Other'),
    ]

    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='files')
    original_filename = models.CharField(max_length=255)
    encrypted_file = models.FileField(upload_to='encrypted/')
    file_type = models.CharField(max_length=50, choices=FILE_TYPES, default='other')
    file_size = models.PositiveBigIntegerField(default=0)
    encryption_algorithm = models.CharField(max_length=50, default='AES-256-GCM')
    salt = models.BinaryField(max_length=32)
    nonce = models.BinaryField(max_length=16)
    uploaded_at = models.DateTimeField(auto_now_add=True)
    description = models.TextField(blank=True)
    download_count = models.PositiveIntegerField(default=0)
```

```
    class Meta:
        ordering = ['-uploaded_at']

    def __str__(self):
        return f"{self.original_filename} ({self.user.username})"

    def get_file_size_display(self):
        size = self.file_size
        if size < 1024:
            return f"{size} B"
        elif size < 1024 ** 2:
            return f"{size / 1024:.1f} KB"
        elif size < 1024 ** 3:
            return f"{size / 1024 ** 2:.1f} MB"
        return f"{size / 1024 ** 3:.1f} GB"


class AuditLog(models.Model):
    ACTION_CHOICES = [
        ('upload', 'File Uploaded'),
        ('download', 'File Downloaded'),
        ('delete', 'File Deleted'),
        ('login', 'User Login'),
        ('logout', 'User Logout'),
        ('register', 'User Registered'),
        ('failed_decrypt', 'Decryption Failed'),
    ]

    user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True)
    action = models.CharField(max_length=50, choices=ACTION_CHOICES)
    file = models.ForeignKey(EncryptedFile, on_delete=models.SET_NULL, null=True, blank=True)
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    timestamp = models.DateTimeField(auto_now_add=True)
    details = models.TextField(blank=True)

    class Meta:
        ordering = ['-timestamp']

    def __str__(self):
        return f"{self.user} - {self.action} at {self.timestamp}"
```

12.2 Critical Settings (settings.py)

Setting	Value / Configuration	Purpose
SECRET_KEY	Environment variable (never hardcoded)	Django CSRF, session, and cookie signing key.
DEBUG	False in production	Prevents stack trace exposure in error responses.
ALLOWED_HOSTS	['yourdomain.com']	Restricts HTTP Host header to prevent host header injection.
DATABASES	SQLite (dev) / PostgreSQL	ORM database backend configuration.

Setting	Value / Configuration	Purpose
	(prod)	
MEDIA_ROOT	BASE_DIR / 'media'	Local filesystem path for encrypted .vault file storage.
DATA_UPLOAD_MAX_MEMORY_SIZE	2621440 (2.5 MB)	Maximum in-memory file upload size.
SESSION_COOKIE_AGE	1209600 (2 weeks)	Session expiry in seconds from last activity.
SESSION_COOKIE_HTTPONLY	True	Prevents JavaScript access to session cookie.
SESSION_COOKIE_SECURE	True (production)	Enforces HTTPS-only cookie transmission.
CSRF_COOKIE_HTTPONLY	True	Prevents JavaScript access to CSRF cookie.
SECURE_SSL_REDIRECT	True (production)	Redirects all HTTP requests to HTTPS.

Settings.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = 'django-insecure-securevault-ecommerce-key-change-in-production'
DEBUG = True
ALLOWED_HOSTS = ['*']

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'vault',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'securevault.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
```

```
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'securevault.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

AUTH_PASSWORD_VALIDATORS = []

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

STATIC_URL = '/static/'
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

LOGIN_URL = '/login/'
LOGIN_REDIRECT_URL = '/dashboard/'
LOGOUT_REDIRECT_URL = '/login/'
```

12.3 AuditLog Model (AuditLog)

Field	Type	Constraints	Description
id	AutoField (INT)	PK, Auto-increment	Unique identifier for each audit log entry.
user	ForeignKey(User)	NULLABLE, SET_NULL	Linked user. SET_NULL preserves logs after user deletion.
action	VARCHAR(50)	NOT NULL, CHOICES	Event type: upload / download / delete / login / logout / register / failed_decrypt.
file	ForeignKey(EncryptedFile)	NULLABLE, SET_NULL	Linked file. SET_NULL preserves logs after file deletion.

Field	Type	Constraints	Description
ip_address	GenericIPAddressField	NULLABLE	Client IP address at time of security event.
timestamp	DateTimeField	NOT NULL, auto_now_add	UTC timestamp of the security event.
details	TextField	NULLABLE, BLANK	Human-readable description (e.g., 'Encrypted orders.csv, 4096 bytes').

Views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, logout
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.http import HttpResponse
from django.core.files.base import ContentFile
from django.db.models import Sum
from cryptography.exceptions import InvalidTag

from .models import EncryptedFile, AuditLog
from .forms import RegisterForm, LoginForm, FileUploadForm, FileDecryptForm
from .encryption import encrypt_file_to_bytes, decrypt_file_from_bytes, get_encryption_info
```

12.4 Production Deployment Configuration

Component	Recommended Stack	Purpose
WSGI Server	Gunicorn (4 workers)	Replaces Django dev server for concurrent request handling.
Reverse Proxy	Nginx	SSL termination, static file serving, rate limiting.
Database	PostgreSQL 15+ with psycopg2	Production-grade multi-user database backend.
File Storage	Local MEDIA_ROOT or AWS S3 (via django-storages)	Encrypted .vault blob storage backend.
Session Backend	Redis via django-redis	Replaces DB sessions for performance under high load.
Secret Management	Environment variables or AWS Secrets Manager	Keeps SECRET_KEY and DB credentials out of source code.
SSL/TLS	Let's Encrypt (Certbot) or commercial certificate	HTTPS enforcement for all traffic.

— End of Document —

SecureVault LLD v1.0 | Yenepoya Institute of Arts, Science, Commerce and Management | 2025-26